Draft version September 3, 2026
Typeset using LaTeX **twocolumn** style in AASTeX631

Can AI Agents Analyze Gamma-Ray Bursts?

Vikas Chand,[1] Khushboo Sharma,[2] and Jagdish C. Joshi[2]

[1] *Louisiana State University, Baton Rouge, LA, USA*
[2] *Aryabhatta Research Institute of Observational Sciences (ARIES), Nainital, India*

ABSTRACT

Time-resolved spectroscopy of gamma-ray bursts (GRBs) rests on judgement calls: which detectors to trust, where the background lies, where the burst begins and ends, how finely to bin it, and which of two dozen spectral models to adopt. We describe the design of an agent that makes those calls under human approval, and what it caught on its first burst. A language model runs inside a harness of deterministic tools, plain-language skill documents, state derived from disk, and enforced boundaries; a producer never verifies its own work. The agent analyses each burst blind, freezes its numbers before it reads the published literature, and turns every mismatch into a numbered lesson with an executable test. Applied to a campaign of 106 single-pulse *Fermi*/GBM bursts, the design fits a 24-model menu to every time bin and assembles reports whose numbers recompute from the products on disk; on the first burst walked end to end it caught engine defects, now fixed and tested, and confirmed blind predictions while refuting one. The trained artifact is the skill library, readable by a student or by another AI system, which we release with the engine, the tests, and the per-burst records.

Keywords: Gamma-ray bursts (629) — Astronomy data analysis (1858) — Astronomy software (1855) — Astrostatistics (1882)

1. INTRODUCTION

The question in our title is usually asked as if it had a yes-or-no answer. It does not, because "AI doing analysis" names at least two different architectures, and the difference between them decides what the answer can mean. In the first architecture, a *workflow*, the analysis follows predefined paths—the same stages, in the same order, for every burst—and software executes them. In the second, an *agent*, a model directs its own process: it decides what to do next, which tools to call, and when it is finished (Anthropic 2024). Workflows are auditable and dull; agents are flexible and hard to trust. Most claims that "AI analyzed the data" blur the two.

This paper builds and tests a deliberate hybrid, and names it honestly: a *gated workflow with bounded agency*. The analysis pipeline—data acquisition, detector and background selection, adaptive binning, spectral fitting, temporal analysis, quality control—is a workflow: its stages are fixed, its statistics are chosen in advance, and its outputs are stamped with provenance. Inside that workflow, an AI exercises agency at exactly the points where judgement is required and we can afford to audit it: which of twenty-four spectral models wins a gated comparison, whether a fit is valid or sits on a parameter rail, whether a published value has been verified at its source, and—when our numbers disagree with the literature—whether the fault is ours, theirs, or a difference of analysis frames. A human approves every step before the next one runs. Nothing advances on silence, and no approval is ever fabricated: every decision carries a stamp naming who, or what, made it.

Why is time-resolved GRB spectroscopy the right testbed rather than a convenient one? Because it concentrates, in one measurement problem, the three features that make scientific AI hard to evaluate. First, the decisions have no answer key. Two careful published analyses of the same burst, using overlapping data and defensible methods, can disagree about whether a thermal component exists at all; the literature is a *distribution* over defensible choices, not a ground truth (Siddique et al. 2022; Fana Dirirsa et al. 2019). Second, the instrument matters: responses age, detectors are occulted, backgrounds are fit, and a systematic that is invisible in one regime of burst brightness or hardness

vikas.chand.physics@gmail.com

appears only when the sample happens to present the other. Third, the stakes are population-level: our science application is a census of spectral shapes across 106 single-pulse bursts selected morphologically (Busby & Lazzati 2024), and a small per-burst bias becomes a population-level artifact. A system that merely *runs* on such data is easy; a system whose outputs deserve trust must show its work at every one of these pressure points.

Our design answer, developed in §3, has one unusual component that we believe is the paper's main methodological contribution. The system's procedural knowledge lives in a *Skill Library*: versioned, plain-language documents—one per pipeline stage—that state how the stage is run, what its known failure modes are, and what was learned from every burst that ever taught it something. The Library is executable culture: an AI agent reads the same document a new student would, and both are bound by the same rules. Around the Library sits the second component: a roster of single-purpose specialist agents—each defined by its role, the skills it reads, and the domain tools it commands (§3.3)—and an operating skeleton of typed states, typed failures, and mechanical gates that makes their fan-out and reassembly auditable (§3.4). Crucially, the Library *learns* under a discipline we call lessons-as-tests: a lesson enters the Library only as a claim with provenance (which burst, which date, which evidence) *and* an executable test that fails if the lesson is ever violated again. Weights are never updated; the harness is. §5 shows what this training regime produced over the first ten bursts of the campaign, including its learning curve, and §6 develops the verification doctrine—blind-first analysis, frozen predictions, and an independence requirement we state as: agreement between two analyses is evidence only in proportion to how deep their shared primitives go.

We are not the first to ask whether AI can do science end-to-end. Multi-agent systems now generate ideas, run analyses, and draft papers across disciplines (Villaescusa-Navarro et al. 2025; Yamada et al. 2025). Our design differs in direction: where those systems are breadth-first—an agent writes fresh analysis code for each project—ours is depth-first: the agents operate a single, tested, domain-specific engine, and the system's growth is the accumulation of verified domain lessons rather than of generated artifacts. The comparison is instructive in both directions, and we return to it in §10: the breadth-first systems demonstrate, at scale, exactly the evaluation gap—qualitative review standing in for ground truth—that our gated, decision-level scoring is designed to close.

The paper is organized as follows. §2 describes the sample and data. §3 presents the architecture: the Campaign Protocol, the Skill Library, the specialist roster and its domain tools, the authority split between workflow, agent, and human, the doctrine distilled from its failures, and the operating skeleton that enforces it. §5 presents harness learning and the campaign learning curve. §6 presents the verification doctrine and the adversarial experiments. §8 describes the human-versus-agent benchmark design. §9 reports walkthrough-era results, all provisional$^{(p)}$ until a frozen production sweep. §10 discusses limits, and §11 concludes. Throughout, walkthrough-era numbers carry the flag$^{(p)}$; they will be regenerated mechanically when the converged pipeline is frozen and run once over the full sample.

2. SAMPLE AND DATA

[PENDING run: port v1 §2 with the L17 inclusion-gate paragraph added]

3. THE HARNESS, COMPONENT BY COMPONENT

We use one vocabulary throughout, and we separate the two kinds of model the paper needs. The *language model* is the model we call; the *spectral models* are the functions we fit to photon counts. We follow the published literature in calling the whole system an *agent*: a language model that acts through tools in a loop toward a goal. A bare language-model call retains no project state between calls and observes nothing outside its context; it can test a claim against the world only through the context and the tools placed around it. We call that surrounding machinery the *harness*. Ours has two layers. A hosted coding product supplies the runtime loop, the tool calls, the compaction of a long conversation, and a permission layer; the repository adds the GRB-specific tools, instructions, state records, and gates. The harness invokes the language model in named *roles*, each with a stated context and tool grant; a *subagent* is a separately invoked, fresh-context role instance that returns a bounded result to the supervising session. So the agent contains a harness, and the harness invokes subagents.

We organise the section by five runtime jobs that any agent harness must perform: run the reasoning loop, execute tools, assemble and manage context, persist state, and enforce boundaries. Two distinctions recur under every job. A control may act before a step, steering what the model is about to do (we call these *guides*), or after it, checking what the model did (*sensors*); and a control may be implemented in code, where it is deterministic and testable, or rest on a model's judgement, where it is not. We built the system before adopting this vocabulary and use it here only to expose the design; the related work of §1 places the design among

published agent architectures. Figure 1 lays our system out under the five jobs using the census convention of the component roster (Table 2, Appendix A): a box is drawn solid when a file or a recorded product for it existed in the working tree on the census day, and dashed when the component is only proposed. The roster states the work of every component in one sentence, with its origin quoted where one is recorded. Tracked code was pinned at one commit$^{(p)}$ for every count in this section; the products the pipeline writes are untracked by design and are bound by content hash, so a count of products names the working tree and its date, not a commit.

3.1. *The loop and the state machine*

The reasoning loop today is the interactive session of the hosted product, which we label Mode B internally: the product supplies the loop, the tool execution, the compaction of long context, and a permission layer, and the repository supplies everything that makes the session a GRB analyst. The analysis of one burst runs through eleven ledger steps, from the literature harvest and the circular record, through data inventory, the three Stage-1 selections and the binning, to spectral fitting, timing, the spectral-energy panels, and the quality-controlled report; §4 walks that lifecycle. The protocol requires each step to run, to be presented to the human as four things (what the step does, what actually ran, the conclusions with their honest flags, and what is unexplained), to be gated, to be compared with the literature where the step calls for it, and to be distilled before the next step starts. It also calls for a *dispatcher* at task intake, which reads the task and the requirements register and returns the agents, gates, and order the task requires, naming every guard that does not yet exist. In the current interactive deployment these invocations are procedural: the session performs them by protocol, and nothing schedules them automatically. The hosted product retains and compacts in-session context; durable scientific state is meant to live on disk. The skeleton document specifies fourteen typed states for a burst, thirteen from registered to bundled plus one terminal exclusion, and a board tool derives a working state index for all catalogued bursts$^{(p)}$ from selected files on disk. The board does not yet implement every declared state, and it labels product currency as unverified rather than asserting it. Failures have typed outcomes; a warning alone does not count as handling (the taxonomy is in §10).

Two parts of the loop are declared debt. The queue manager that would own campaign sequencing, hold the memory-budget claims, and resume after a kill, and the eleven workflow files$^{(p)}$ that would run the transitions from binning to presentation, the release, and invalidation as fixed sequences of reader, producers, verifiers, and stamps, are specified in the skeleton and not built. The corresponding computational stages run today on prototype shell and Python chains that invoke no agent. Both are drawn dashed, and §10 returns to what their absence costs.

3.2. *Tools: the deterministic producers*

The tools are ordinary, testable code, and the agents command them through a deliberately small interface: nine of the ten agent definitions carry only file reading, searching, and a shell, and the tenth adds file editing so that it can write lessons. Six domain instruments do the science. The spectral engine defines a menu of twenty-four models$^{(p)}$ (six default, two shape variants, sixteen high-energy extensions); in the campaign configuration it attempts that menu for every usable time bin and for the integrated interval when its detector plugins can be built, and it writes per-model status, validity, and information criteria together with row-level bound-pinning diagnostics. The adaptive binning tool forms Bayesian blocks from the catalog-brightest approved detector, or otherwise from the approved detector with the most counts between its own background windows, and merges blocks below a significance floor using the fitting library's own classes. The Stage-1 approval tool shows the light curves and records a human or vision-agent decision on detectors, background windows, and source window, with a stamp naming who approved and how. The temporal chain measures durations, the minimum variability timescale, and the spectral lag from the same approved selections; it imports the lag routine of an earlier project directly rather than porting it, and the reproduction record must pin the imported file's revision. The paper-grade spectral-energy producer renders each usable bin under the display semantics of the standard X-ray fitting package, retries a drifted live solution as a frozen replay of the stored parameters, and refuses the panel if neither reproduces the stored information criterion within a tenth; the legacy and montage paths stamp or display a mismatch instead. The report assembler draws on the burst's own products and prefers the promoted fit table, but it does not yet fail closed when that table is absent, nor bind every report to the table's hash. The consequence these guards protect is concrete: an invalid or drifted fit must not become a population count. The fitting step itself, where most of the bounded agency lives, is described with its own figure in §4.

3.3. *Context: skills, checklists, and memory*

An operational specialist in this system is a role contract, the skill material that role must read, and a tool

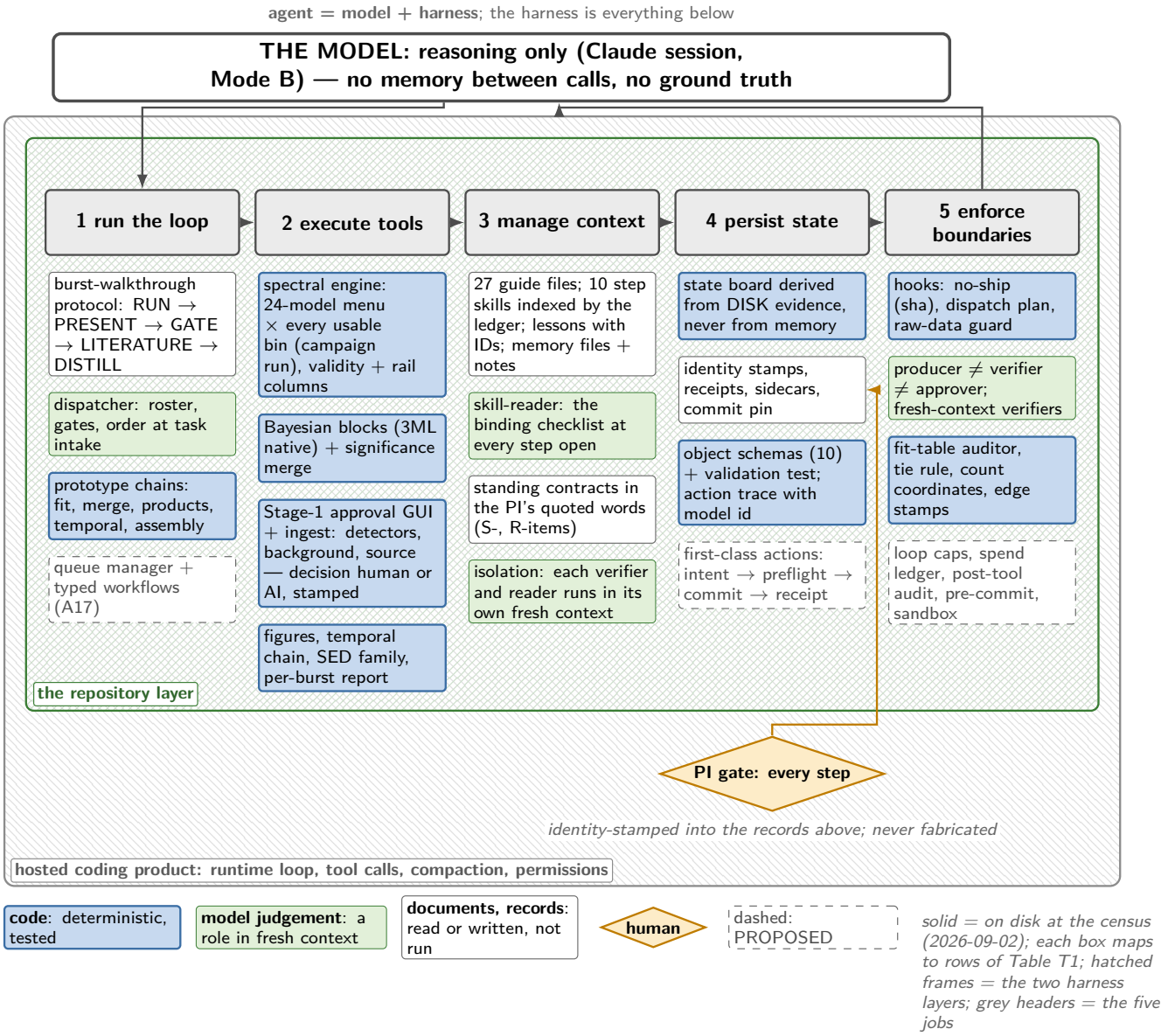

Figure 1. The harness anatomy. The five runtime jobs of an agent harness, with the components this campaign has under each. Colour says what executes: blue boxes are deterministic code, green boxes are roles the language model plays in a fresh context, white boxes are documents and records that are read or written but not run, and the amber diamond is the human. Solid boxes satisfy the roster's file-or-recorded-product convention in the census working tree; dashed boxes are proposed in the requirements register and not built. The two hatched frames are the two harness layers: the hosted coding product outside, and inside it the repository layer, which is what makes the session a GRB analyst. The protocol requires every gate to carry an approver identity; stamp coverage is not yet complete across the sample, and Stage-1 approvals may be human or AI, stamped either way. Each box maps to rows of Table 2.

grant; its specialisation lies in interpreting domain outputs, not in invoking code. Context is what the language model sees when it acts in such a role, and the harness's job is to supply the right context at every step without letting a long run crowd out what matters. Three moves do this. *Reduce*: the hosted product compacts old conversation, and the per-burst live report is rebuilt from stamps rather than carried in prose.

Offload: durable guidance lives in twenty-seven guide files$^{(p)}$ under one directory, ten of them step skills indexed by the burst ledger, each carrying the step's criteria and checklist, and five of those also carrying numbered lessons under a prefix unique to the file; a small set of standing contracts, written in the human's quoted words, fixes what every figure and report must satisfy. The intended skill for the spectral-energy step is not yet

written. A small set of standing contracts, written in the human's quoted words, fixes what every figure and report must satisfy. *Isolate*: the protocol requires every verification to run in a fresh context that has seen nothing of the producer's reasoning, and inventories of the repository are delegated to subagents that return conclusions rather than file dumps. Fresh contexts are correlated error-detection channels, not independent reviewers: they share the model family and the primitives, and §6 states what independence requires. The *skill reader* turns the offloaded law into action: at each step opening the protocol calls it to read the step's skill document, its defect ledger, and the open register rows, and to return the binding checklist for *this* burst. It produces nothing else, and its invocation and the downstream audit of checklist compliance await the typed workflows. Cross-session memory is a small indexed file of durable facts that the fresh-session protocol instructs the operator to read at the start of every session and to consolidate at its end.

3.4. *State: derived from disk, stamped, receipted*

The system may claim only what it can evidence. The board computes a working status index for the catalogued bursts from selected files on disk, so it is useful for finding absent evidence, but it does not yet prove the currency of what it finds. The protocol requires every step gate to leave an approval stamp naming the approver, the time, and the status, and the live report links the stamps and products it knows how to locate; one burst carries a stamp for every step it has passed so far (steps 0b to 5 approved, step 6 presented), the walkthrough burst of §9. Approvals are revocable. An operator-invoked invalidation tool, dry-run by default, marks every later approval stale when an upstream step changes and clears the downstream completion markers; it does not yet encode a dependency graph. The protocol requires evidence before reinstatement. Both directions have operated on the walkthrough burst, and its approval record carries them. When the background windows of two detectors were trimmed after steps 3 and 4 had been approved, the cascade demoted the later approved step and, finding no stamp at step 4 to demote, exposed that the step-4 approval had been given in conversation but never stamped; the record now carries that stamp, dated to the conversational approval, with the late recording disclosed on the stamp itself. The demoted binning approval was reinstated only on proof that the binning's reference detector was untouched by the trim and its block table byte-identical, the table's hash being written into the reinstatement note.

Products bind to their inputs by content hash and to their generators by commit, for the products so far covered: promotion receipts record the hashes of the staged and promoted fit tables for six bursts(p), a sidecar beside each promoted table records the detectors, ranges, and convention in force, and a campaign commit pin records the intended generator revision. Recording the full command line and environment, and binding every report to its fit table's hash, remain proposed. Since the census day, ten JSON schemas(p) cover nine of the object types the pipeline writes and the per-burst approvals file that holds the stamps; the schema test validates the seven classes with instances on disk and the action trace, and ledgers four frozen records that pre-date the contract. Its first run found the literature-harvest manifests spelling their paper list under four different keys and two malformed bibliographic codes, which is what a schema is for; validation at write time for new manifests remains proposed. One honesty item remains: products live in two roots resolved by an index file, so the system does not yet have one reality, and the object-and-action model of §7 says what closing that gap requires.

3.5. *Boundaries: hooks, roles, and the human gate*

The design principle is to place each boundary at the strongest layer that can hold it; the controls in place range from hooks to role prompts to prose, with the gaps stated. Three repository hooks(p) inspect selected tool calls before they run. One checks any image bound for the human against the hashes in the verification ledger and refuses an unverified one; one checks a pattern-selected subset of shell commands that launch pipeline producers for a dispatch plan younger than a day; and one rejects pattern-matched commands that would delete or overwrite raw data, with acquisition commands exempt. Their limits are stated as plainly as their powers. The no-ship hook sees only PNG files; the dispatch hook covers one of the five launch paths of the products chain that the register enumerates(p) and matches the text of a command rather than what it executes, so it has blocked commands, some of them read-only, that merely named a producer; and the raw-data guard is a pattern, not a filesystem jail. Outside those matchers, control passes to fresh-context agents and, weakest of all, to prose, and several planned boundaries are not yet implemented.

The protocol assigns production, verification, and approval to separate roles: the producer of an artifact never verifies it, the verifier never approves it. The typed workflows would enforce that separation structurally; the current interactive deployment invokes it procedurally. Four kinds of actor are admitted to the

action record: the human, who approves and rules; the agent, whose identity and model are stamped on what it presents, verifies, or distils; the hook, which refuses; and the script, which produces and writes its receipt. Automatic capture of every tool call is not yet wired, and an identity the session cannot establish is recorded as unknown rather than guessed.

Two planes organise who may say what. On the *production plane* a burst runs the fixed protocol and its numbers may enter a table only after the gates of §4. On the *discovery plane* an agent may explore, post-select, and propose; nothing found there is a result until it is promoted to the production plane through a stamped gate with its evidence, which is the human's decision. Table 1 states the split.

Table 1 states the paper's honesty contract. The *workflow* and the deterministic *engine* decide everything fixed in advance: the step order, the energy selections, the statistic, the multistart seeds, the physical-validity flag of every fit, and the validity-gated ranking by information criterion. The *AI* works within audited bounds: it audits those outputs, verifies a published value at its source and aligns its frame, proposes how a mismatch is attributed, drafts lessons, and escalates what it cannot resolve. The *human* decides everything consequential: the scope, every step gate, whether a proposed lesson enters the library, when convergence is declared, and every scientific claim that leaves the system. In a fully AI-run report the documented protocol assigns the intermediate gates to an independent non-producer AI; the identity that would approve in that mode is a decision the human has not yet taken, and we list it as pending rather than assume it. The protocol forbids fabricating a human approval, and the stamping tool refuses an approval without a supplied identity. The human has overruled the agent's proposal on the record; §9 reports that count with its denominator rather than characterising it here.

What the roster makes visible, finally, is the shape of the harness as it stands. Under the roster's convention, forty-four of forty-nine rows$^{(p)}$ are deployed and five are proposed, and several deployed rows remain partial, as their status notes record. The five proposed rows are the queue manager with its typed workflows, the first-class action wrappers, the report-conformance gate, the queued boundary package (loop caps, a spend ledger, further hook points, pre-commit checks, and a sandbox), and the queued science guards. Each would turn a protocol into a mechanism. We return to this pattern, and to what it says about where a scientific harness should grow next, in §10.

4. GUIDES AND SENSORS ALONG THE BURST

Section 3 took the harness apart by job. This section follows one burst through it in time. The unit of work is one burst walked through eleven ledger steps, and at every step the harness does two things besides run the step: it steers the model before the step with *guides*, the skill document, the checklist, and the standing contracts that apply, and it checks the result after the step with *sensors*, some of them code and some of them fresh-context roles. Figure 2 draws the ledger with the guides on the left and the sensors on the right; blue sensors are code, green ones are model judgement, and the amber diamond between them is the human gate that every step passes before the next begins.

4.1. *The eleven steps*

The ledger begins before the data. The *literature harvest* (step 0b) finds and fetches every refereed paper that analysed the burst, using at least four query forms (the GRB name, the trigger name, the day fraction, the trigger number), preferring the version of record, and recording each harvested value with its conventions: which peak-energy definition, which sign convention for the lag, which energy band, which time interval on whose clock, which detectors. The harvest is taken early and compared late; the comparison itself waits for step 9, after our own numbers are frozen (§6). The *identity and circulars* step (0) resolves which burst this actually is, since same-day triggers exist, and reads every real-time circular; a burst with no circular is recorded as a result, not as a gap. The *data inventory* (1) verifies that the downloaded data can support the analysis: file versions, response-matrix time coverage against the source window, and detector geometry, including the check that no well-placed detector is occulted by the spacecraft, a failure invisible to pointing angles alone. The three *selection* steps (2–4: detectors, background windows, source window) are decided by a human in a purpose-built interface or by a vision-capable role, and either way recorded with a stamp naming the decision-maker and the mode; downstream the pipeline treats those stamps as immutable inputs, and the walk-through presents the recorded decision rather than re-adjudicating it. The *adaptive binning* step (5) derives Bayesian-block time bins from the reference detector and merges them to a significance floor. *Spectral fitting* (6), where most of the bounded agency lives, has its own subsection below. The *temporal* step (7) measures the duration, the minimum variability timescale, and the

Table 1. The authority split: who decides what.

the workflow and the engine decide (fixed in advance)	the AI does (bounded, audited)	the human decides (gates)
step order and protocol	audits the code-computed comparison; frames the supported interpretation	scope and completion contract
energy selections; statistic	checks the validity outputs; flags anomalies	Stage-1 selection in the human arm; acceptance of the AI arm's selection
multistart seeds; bound gates	literature verification and frame alignment	every step gate
physical-validity flags; validity-gated ranking	mismatch attribution; lesson drafting	lesson acceptance; convergence; freeze
provenance stamping; typed outcomes	escalation of what it cannot resolve	promotion from the discovery plane; every claim

spectral lag from the same approved selections, with the estimator named on every value and the lag sign convention stated. The *spectral-energy panels* (8) render every bin under the display semantics of the standard X-ray fitting package and are read residual by residual. *Quality control* (9) assembles the report, runs the literature comparison, and closes the burst with a typed verdict and named flags. A bin in which no model passes the validity gate is declared inconclusive by the engine, and a burst whose response does not cover its source window is excluded structurally; both are recorded, *successful* outcomes. The workflow is forbidden to keep adjusting an analysis until it manufactures a preference.

The guides are the same kind of object at every step: the step's skill document, read end to end by the skill reader before the step opens, with its criteria, its checklist, and, where the step has accumulated them, its numbered lessons; for the selection steps the human's rulings quoted verbatim; for the figure and report steps the standing contracts. The sensors differ by step because the artifacts differ. A harvest manifest is validated against its schema. An inventory figure passes the figure verifier and its numbers are recomputed against the sample catalog. A stamped selection is validated by the catalog validator, and the state board records the block table before fitting may start. A fit table passes the engine's own guards, the fit-table auditor (the argmin, the rails, the tie set, the count coordinates), the lessons-as-tests, and, on presentation, the tie reporter and the numbers verifier. A temporal product passes the figure and numbers verifiers, the seed auditor, which replays any stochastic product from its recorded seed, and the port verifier where code was ported. A panel cannot reach the human unless its hash is in the verification ledger, and every panel is read by the figure verifier. The report passes the numbers verifier and the blind-first literature comparison; a report-conformance gate that would check the report against its contract as code is specified and proposed. Beneath the ledger two checks run continuously: the evaluation battery, one command that runs the test suite and the fit-table auditor over every promoted table and records, with its result, the model identity it was given and the commit, and the distillation of every incident into a lesson and a register row in the session that produced it (§5).

4.2. *Inside the fitting step*

The fitting engine confronts each time bin with twenty-four spectral models: six base models, two free-smoothness variants, and sixteen high-energy extensions (Fig. 3). Three rules govern what the numbers may mean. First, *inclusion is gated on data quality, never on detection significance*: every detector band with usable data and a valid response enters the joint fit, because conditioning inclusion on significance manufactures spurious detections in exactly the bursts where a band fluctuated upward. Second, *every model family receives multistarts*: unconditional for the continua and the two-break family, and triggered for the composites whenever a component rails, its nested test is weak, or the fit is worse than its simpler parent, because a chained seed from a bright epoch can strand a faint bin in a wrong local minimum, a failure we document in §5. Third, *validity gates* exclude from model selection any fit whose shape parameters sit on a bound, with the margins computed in the parameter's natural geometry (logarithmic for energy-like parameters spanning decades), a rule the campaign learned from its first soft burst.

Model comparison is then reported as evidence, not verdicts. The engine ranks the valid fits by the Akaike information criterion and writes the winner; the interpretation quotes the winner's margin twice, against the runner-up (*which* model) and against the best simpler model it nests (*is the extra component needed at all*), each converted to an evidence ratio $e^{\Delta\mathrm{AIC}/2}$ and graded. A margin above 6 corresponds to more than roughly 20:1 and is tracked as strong; at or above 10, roughly 150:1,

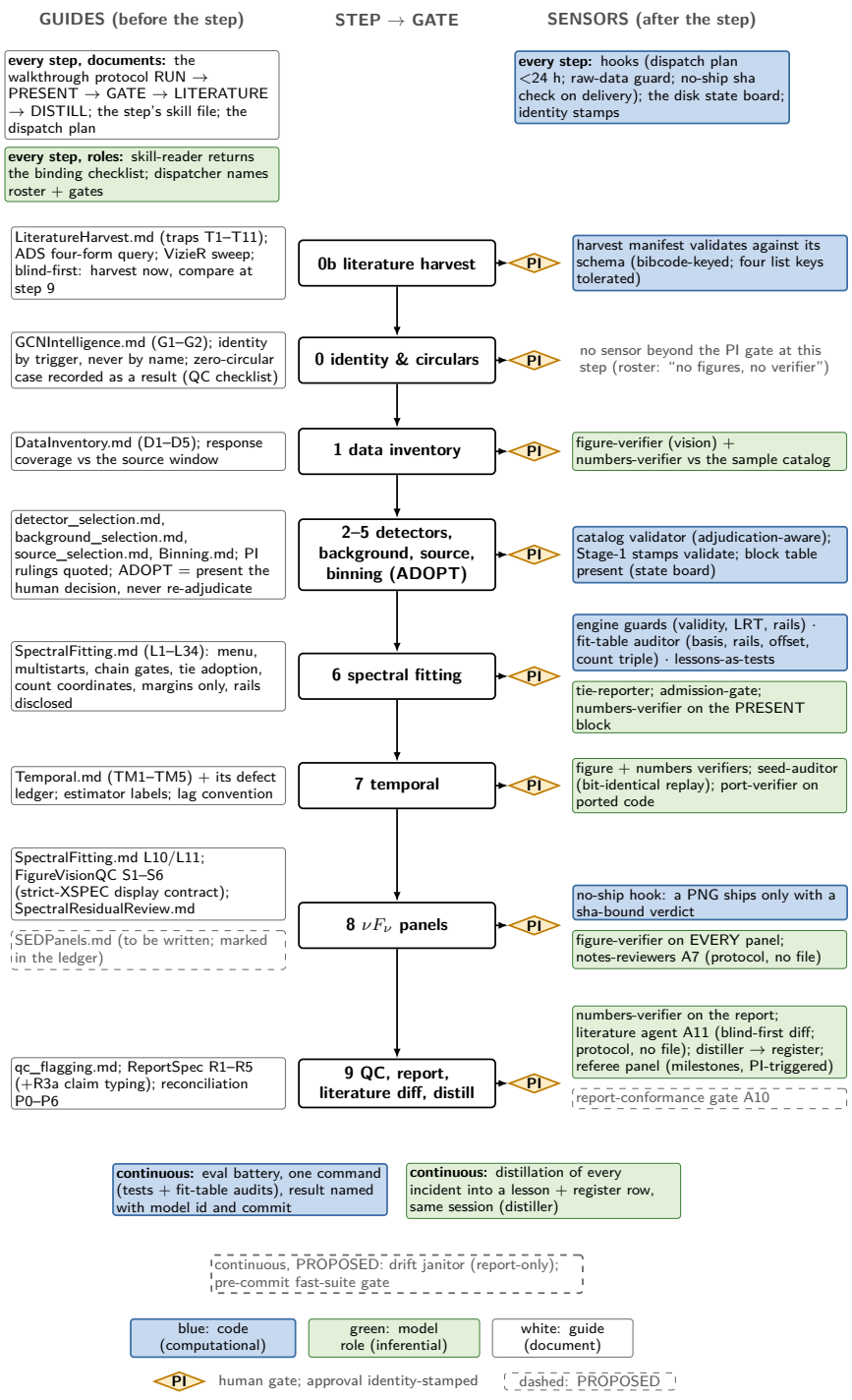

Figure 2. The per-burst lifecycle with its guides and sensors. Centre: the eleven ledger steps, top to bottom, each followed by the human gate. Left: the guides that steer each step before it runs, chiefly the step's skill document with its numbered lessons and the standing contracts in the human's words. Right: the sensors that check each step after it runs, blue where the check is code (tests, hooks, the fit-table auditor, schema validation) and green where it is a fresh-context role (figure, numbers, seed, port verifiers; the tie reporter; the admission gate); guides are white, and the amber diamond is the human gate. Below the ledger, the checks that run continuously. Dashed items are proposed.

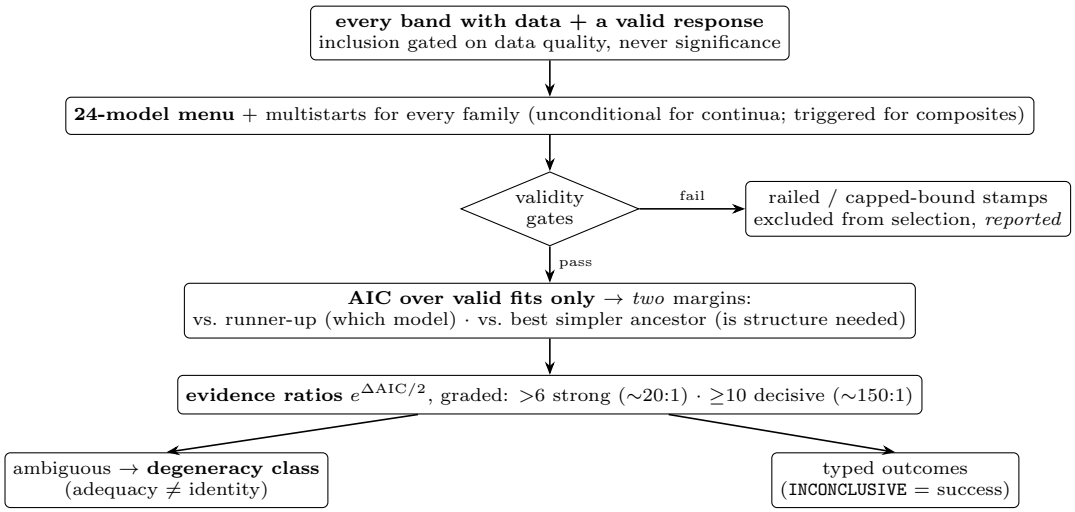

Figure 3. Inside the fitting step. The engine is deterministic; the agent's bounded agency is the audited interpretation of its outputs.

decisive; anything less is reported as what it is. Heads within two units of each other are a tie, never a winner: the tie is reported as a set, and where one model must be adopted for a downstream product the adoption rule prefers the fewest parameters and then the lowest criterion, and says so. Where extra structure is required but thermal and non-thermal descriptions sit within a few units of each other, the bin is assigned to a degeneracy class rather than a physics claim: *adequacy and identity are different questions*, and the second frequently cannot be answered from these data alone. Two further stamps ride on the artifact rather than in prose: a bound-pinned parameter is flagged on the fit row where it sits, and the audit record beside the table classes every thermal component by where its peak, at $3.92\,kT$, falls: below the detector band, inside it but below the 20 keV trust floor (edge-constrained), marginal, or in band, so that a reader of the table sees the condition without reading the report.

4.3. *Fan-out and the gates*

Producers fan out: the fitting engine over time bins, the panel builders over models, the harvest readers over the published literature. Their outputs converge on verifier gates, each a fresh-context role with one duty (Table 2). Every figure passes a vision check before any human sees it. Every printed number is recomputed from the run's own products. A model-selection margin below the tie threshold is reported as a tie. A stochastic product must rerun bit-identically from its recorded seed. A ported algorithm is compared numerically with its source before it is trusted. No row enters a committed catalog unscreened. Nothing is re-derived that the project has already proven. Every incident is closed into a lesson, at the correct enforcement layer, in the same session that produced it. A verdict is bound to a content hash of the artifact it certifies and expires the moment that artifact changes. The assembly is the reverse of the fan-out: verified outputs are composed into the burst's report, the burst's typed state advances only when the products on disk evidence it, and the human's approval of step 9 closes the burst.

Two disciplines that wrap the lifecycle belong to later sections. Before any comparison with the literature, our own numbers are frozen, and every mismatch is attributed to our side, to the published side, or to a difference of frame; that protocol is the verification doctrine of §6. And the lifecycle's exit condition is convergence: when K consecutive bursts complete without a new lesson, the skill library is declared converged, the pipeline is frozen at a recorded commit, and the whole sample is re-analysed once by the frozen system; that steering loop is §5. Only the frozen sweep produces citable numbers, and every value in this paper from the walkthrough era carries the provisional flag$^{(p)}$.

5. LEARNING WITHOUT GRADIENTS

Contemporary AI agents are improved in five ways: pretraining on large corpora; supervised fine-tuning on demonstration trajectories; preference optimization against human or model judgments; reinforcement learning on *verifiable* rewards, where a trajectory is reinforced only if its outcome can be checked mechanically (code passing tests, proofs verifying); and *harness-side learning*, in which the model's weights are never touched and the surrounding harness—its instructions, procedures, and memory—accumulates capability instead (Guo et al. 2025; Wang et al. 2023; Shinn et al. 2023). Our campaign is training of the fifth kind carrying the fourth kind's essential ingredient. The weights of

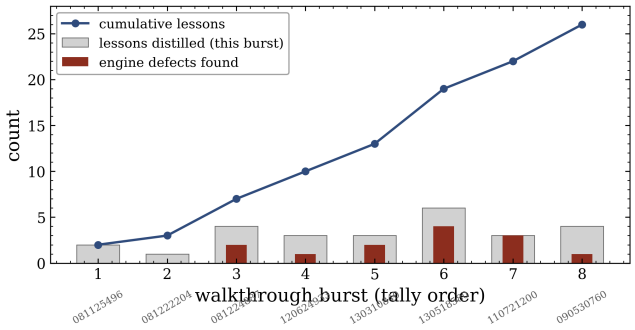

Figure 4. The campaign learning curve[(p)]: lessons distilled per burst (bars), engine defects found (dark bars), and the cumulative lesson count (line) across the first eight walked bursts. The curve is visibly unconverged; its flattening, not a preset burst count, terminates the training phase.

every model we use are frozen; the artifact that learns is the Skill Library; and the unit of learning is constrained to be *verifiable*: a lesson enters the Library only as a claim with provenance (which burst, which date, which evidence) together with an executable test, added to the Lesson Test-Suite, that fails if the lesson is ever violated again. The suite runs against the current analysis products; where a table predates a fix, the corresponding failure is registered in a known-stale ledger with its clearing condition rather than silenced—debt is visible, never normalized.

This training style has properties that gradient methods cannot offer at our scale, and one property they share. It is data-efficient: thirty-two numbered lessons[(p)] from the walked-burst campaign so far. It is auditable: the learned policy is markdown, and every rule cites the burst that paid for it. It transfers across model families: the same Library is executed by different vendors' agents, which is the basis of the multi-system benchmark of §8. It survives model upgrades. And—the shared property—it exhibits a learning curve (Fig. 4), whose flattening is the campaign's stopping criterion: when K consecutive bursts produce no new lesson, the Library is converged and the pipeline is frozen.

Two episodes illustrate what the loop learns and how. The first is a defect we designate L27. The engine's validity gates flagged a fitted parameter as railed when it sat within a fixed *linear* margin of its bound. For energy-like parameters bounded across three decades, that linear margin is an absolute dead zone of tens of keV above the lower bound—and every burst analyzed in the campaign's first seven walkthroughs had a peak energy far above it. The eighth burst was soft ($E_{\rm p} \sim 50$–140 keV[(p)]), and the gate silently disqualified *every* one-break continuum in most of its time bins, delivering the blocks to thermal-composite models by forfeit. Uncorrected, the census would have manufactured a population-level correlation between spectral softness and thermal components—a selection bias wearing the costume of physics. The repair (margins computed in the parameter's natural geometry), its unit test, and the burst's refit happened within the same session; the pre/post contrast—most blocks "preferring" extra structure before, none after[(p)]—is the measured size of the bias the lesson prevents. The episode is also an argument about curricula: the defect was invisible until the sample's serial ordering happened to deliver a soft burst, so *regime coverage, not burst count, is what a training sample must supply.*

The second episode is a cross-validation. Our temporal chain carried a known, ledgered defect: a spectral-lag sign convention inverted with respect to the field's standard. The eighth burst has a published lag of $+7.08 \pm 0.45$ s in the conventional sign (Lu et al. 2018); our chain returned -5.99 s[(p)]. Same burst, same physical direction, opposite sign: the ledgered inversion was confirmed against external ground truth in the course of ordinary operations, and the lesson (state the convention with every quoted lag; validate the sign against a burst with a published direction) was distilled with the defect still unfixed at source—the Library records what is *known*, including known-broken.

5.1. *Divergence learning*

Failures are not the only teacher. Where the human's decision and the agent's proposal diverge and both are defensible, the divergence itself is the signal, and it is processed under a fixed protocol: detect it; elicit the human's reason *in their own words* (a guessed rationale is forbidden—if the human cannot recall, the record says so); validate the stated reason against the primitives; generalize it only through a measured census over the sample, never from the single case; and enter it in a *Divergence Ledger*. The founding case is a detector-selection divergence: the written geometric rule and two independent agent passes selected seven detectors[(p)]; the human had selected four[(p)]. The human's recollected reason—keep the detectors that triggered the instrument, plus the companion detector on the same spacecraft side—validated exactly against the mission's trigger record, and reproduces the detector choice of every published analysis of that burst we harvested. A census of the full sample[(p)] then measured how much of expert practice that candidate rule explains: about eighty percent[(p)], with the remainder requiring judgment the written rule does not capture. The written selection rule therefore stands unamended, and the divergence is recorded as a *measurement of practice*, not promoted to

a decision rule. The same census exposed a contract gap—*none* of the 105 human selection records$^{(p)}$ carried a contemporaneous reason; the single filled record is a retroactive reconstruction and is stamped as such—so rationale capture at decision time is now a registered requirement, while the historical blanks stay blank: a reconstructed reason is never a contemporaneous record. The Ledger gives the paper's closing question its instrument: the convergence rate—how often the agent's proposal already matches the human's decision—is the measurable readiness criterion for ever moving a human gate to an independent approver.

6. VERIFICATION DOCTRINE

6.1. *Blind-first analysis with frozen predictions*

Before each burst is fitted, the Literature Harvest yields a set of falsifiable predictions—derived from published analyses, never from our own outputs—which are frozen alongside a commitment that the fit will not be steered toward them. The comparison happens only after both sides are immutable. The campaign's scorecards make the point better than argument. For the eighth burst, the literature predicted a positive low-energy photon index early in the burst; our blind fit, with independent binning and a far larger model menu, reproduced it (α up to $+0.6^{(p)}$). It predicted monotonic hard-to-soft evolution; our blind E_p track declined monotonically$^{(p)}$. But the natural corollary—that a spectrum this hard should demand a thermal component—was *refuted*: with the L27-repaired gates, no time bin required extra structure at even the "strong" evidence grade$^{(p)}$. A framework that can refute its own frozen expectation is demonstrating the property that matters; confirmation alone demonstrates nothing.

Beyond per-burst predictions, each Burst Record closes against a battery of *known results*: published spectral-lag values with their sign conventions, published Bayesian-block edges, published variability-timescale limits, low-energy photon-index tracks, and—where an instrument we do not fit has published one—polarization constraints entered as external predictions. The battery is how the campaign tests known GRB results and relations as it goes: agreement validates the chain that produced our number, disagreement is pushed through the attribution of §3.1, and either outcome is banked in the Record.

6.2. *Independence lives at the primitive*

Twice in the campaign, two analyses agreed closely and both were wrong or both unnecessary: our fitted blackbody temperature agreed with a published value to five percent while both components were statistically unrequired; and in the experiment below, two independent review channels built the same load-bearing argument on the same silently symmetric parameterization, corroborating each other into a void conclusion. We therefore state the doctrine as: *agreement between two analyses is evidence only in proportion to the depth of the primitives they do not share*—minimizer, table, column, response model, prior document. N agents reading the same inputs are one opinion wearing N hats.

6.3. *The four-channel experiment: generators, not adjudicators*

We tested whether a panel of AI reviewers could adjudicate a genuinely contested scientific question—a burst for which two published analyses of nearly the same data disagree about a thermal component. Four reviewer channels examined disjoint evidence primitives: the spectral-panel *images* only; the numeric *fit tables* only; the block-to-block *evolution tracks* only; and an *adversary* with full access, briefed to refute the component honestly and permitted to run its own diagnostic refits. A separately-briefed adjudicator then audited all four.

The outcome was double-edged. As *generators*, the channels were excellent: the image channel found a figure-construction defect invisible in any table; the adversary localized a cross-normalization anomaly that we later identified—via the published literature—as a physically occulted detector, and demoted a "decisive" margin to an artifact of a railed reference model, both findings surviving audit and entering the engine as permanent stamps. As *adjudicators*, the channels failed exactly as the doctrine predicts: the two numeric channels derived identical margin ladders and returned opposite verdicts (the disagreement was doctrinal, not evidential); the panel's one unanimous conclusion was traceable verbatim to a rule in the shared Skill Library—prescribed unanimity, not measurement; and one channel confabulated statistics that do not appear in its own assigned figure, caught only because the adjudicator reopened the figure. Two corollaries entered the doctrine. First, *verify the adversary too*: the adversary claimed a nondeterministic fitting step; five identical reruns showed spread exactly zero$^{(p)}$—its variance had been its own varying invocations. Second, with/without-data refits are attribution diagnostics only; a data-dropped fit is never evidence for a claim (§6).

7. THE OBJECT-AND-ACTION MODEL

A long campaign accumulates objects: selections, block tables, fit tables, panels, reports, and the records about them. The harness of §3 works only if those

objects are typed, if the links between them are declared rather than remembered, and if the operations that change them leave a record. Figure 5 draws the model as it exists on disk, in four parts: the objects, the links between them, the actions that create or change them, and the roles allowed to act.

Objects.—Thirteen object types are drawn. Five form the burst's own chain: the *burst* (trigger identifier and catalog row), the *selection* (detectors, background and source windows, with the approver and mode stamped on it), the *block table* (bin edges and significances), the *fit row* (one per model per bin: parameters, validity flag, information criteria, effective-area corrections, rail diagnostics), and the *product* (figure, spectral-energy panel, table, or report). Four are records about products: the *approval stamp* (step, status, approver, time, feedback), the *promotion receipt* (hashes of the staged and promoted tables and the validation that passed), the *audit or verdict* (the fit-table audit bound to the table's hash; the vision and numbers verdicts), and the *action event* (actor, model identity, input hashes, verdict, cost). Three are the law: the *register row* (where a guard lives, the incident that bore it, its status and keep-condition), the *lesson* (a numbered entry in a skill file), and the *burst state* (one of the fourteen states of §3.1, derived from disk), together with the *commit pin* that fixes generators by commit and products by hash. Ten JSON schemas$^{(p)}$ cover nine object types and the per-burst approvals file that holds the stamps; five of the thirteen drawn here are among them (the stamp, the receipt, the audit, the action event, and the burst state), and the others are the literature-harvest manifest and its paper records, the frozen prediction record of §6, and the fit sidecar. The test suite validates every instance on disk against its schema. Schemas came late: the pipeline had written these objects for months before their shape was declared, and the first validation pass found four spellings of one key in the literature-harvest manifests. A schema written before the first instance would have cost less.

Links.—A link is declared when a field of one object names another: the trigger key that ties a selection to its burst and a block table to its selection; the block indices that tie fit rows to blocks; the table hash that ties a receipt and an audit to the fit table they certify; the lesson identifier that a register row cites, guarded by a test that refuses duplicate identifiers. Links that hold only by convention are drawn dashed: a stamp finds its selection by trigger and step number, a product finds its fit rows through the stored information criteria rather than a recorded hash, a burst state points to the verdicts it summarises by a flag, and the commit pin relates to the action trace only through the git head each event records. The distinction matters operationally. A declared link can be checked by code; a conventional link can only be checked by a reader who knows the convention. Declaring the remaining links, chiefly the fit-table hash on every product, is the smallest step that would let the report-conformance gate of Table 2 be written as code.

Actions.—An action is an operation that changes what the campaign may claim. Four are first-class on disk today, each leaving a typed record with an identity: *approve* writes a stamp; *present* writes a stamp; *promote* writes a receipt and *quarantine* a manifest; and *invalidate* demotes the approvals downstream of an amended step and records which step changed. Two are proposed and drawn dashed. *Launch producer* would wrap every pipeline run as intent, preflight, commit, and receipt, so that a producer could not start without the plan and could not finish without the sidecar; today the dispatch hook of §3.5 enforces the first half for the transitions it covers. *Deliver* would record every artifact handed to the human; today the no-ship hook refuses an unverified figure but writes no record of what it allowed. The action trace that would hold all six is on disk with its schema, appended by protocol; automatic capture of every tool call is the boundary item listed in Table 2.

Roles.—Four kinds of actor appear on actions, and the model fixes what each may do. The *human* approves and rules, and no approval is ever written on the human's behalf. The *agent*, stamped with its model identity, presents, verifies, and distils. The *hook* refuses. The *script* produces and writes its receipt and sidecar. The separation of producer, verifier, and approver in §3.5 is this role band read as a rule.

One reality.—The model has one structural gap, drawn in the legend rather than hidden: products live in two roots, resolved by an index file, so the same burst can have a fit table in each. The objects themselves do not say which is canonical; the index does, and the promotion receipt certifies the promoted one. Closing the gap means either a single root or a canonical-table field on the burst object, and it is the precondition for the state board to derive every one of its fourteen states from declared links alone. We list it here because the object model, more than any agent, is what would let a second group run this campaign and arrive at the same tables.

8. EXPERIMENTAL DESIGN: HUMAN VERSUS AGENT

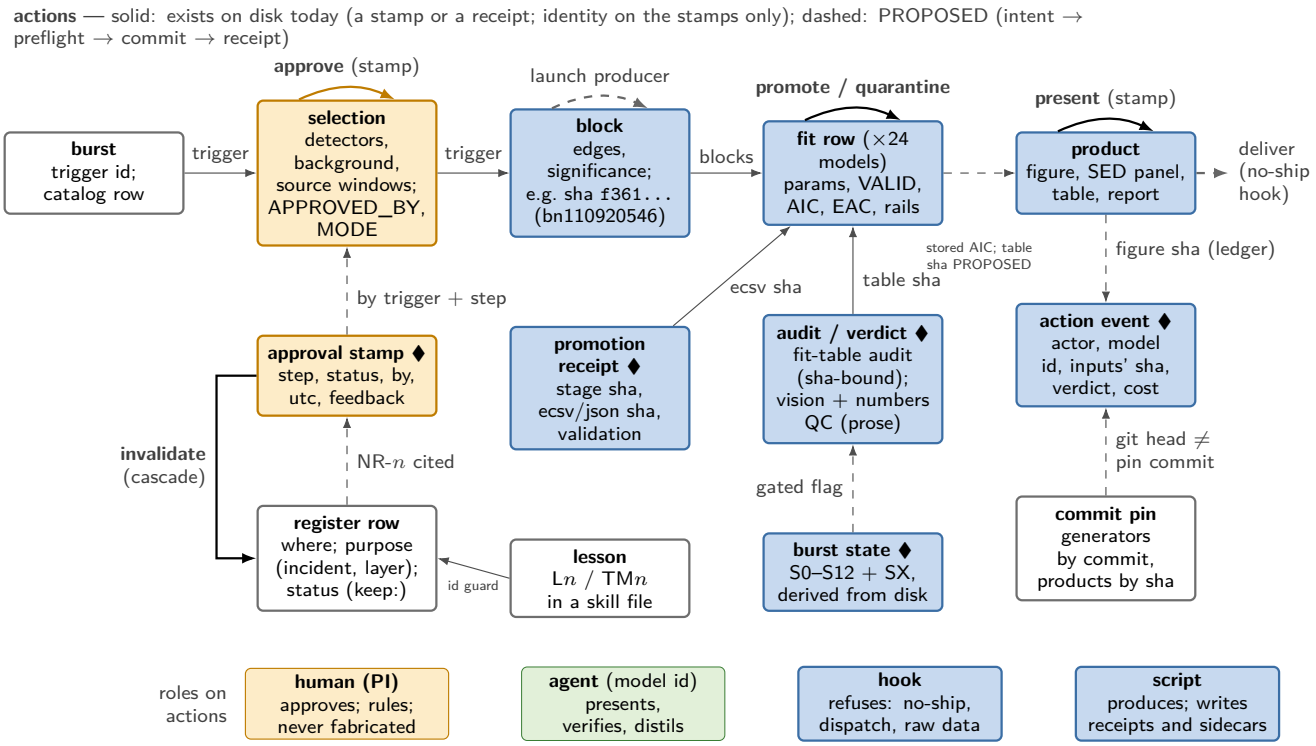

Figure 5. The object-and-action model. Top row: the burst's chain of products, from trigger to report. Middle row: the records about those products. Bottom row: the law that governs them. Colour says who writes the object: blue by code (the audit box also holds the agent-written vision and numbers verdicts), amber carrying an approver identity (human on 433 of the 436 selection rows), white the law and documents; the roles band uses the same colours, green for the agent. A diamond marks an object type with a JSON schema on disk. Links are solid where a declared field carries them (a trigger key, a content hash, a commit) and dashed where they hold by naming convention. Actions are labelled arrows, solid where they are first-class on disk today and dashed where proposed. The band beneath names the four roles that act. Every box maps to rows of Table 2.

The benchmark design retains v1's core: a 25-burst subset receives independent Stage-1 selections from experienced researchers, and agents from multiple vendors—executing the *same* Skill Library, so that differences isolate the model rather than the prompt—are scored not on matching any single expert but on landing within the *spread of the experts*, decision by decision: detector sets, background windows, source intervals, model verdicts. Scoring decisions rather than published numbers is forced on us by the field itself: published analyses of the same burst disagree, so the literature is a distribution over defensible choices and cannot serve as an answer key. **[PENDING run: the multi-system benchmark run; a second independent expert completes the human spread]**

We add two further controls in operations. The walk-through queue runs in two lanes—an older-era lane and a recent-era lane (2021–2025)—so that any skill, constant, or contract that works in one era and fails in the other is exposed as a cross-era defect rather than absorbed as noise. In parallel, a human collaborator replicates the analysis with the Stage-1 selections held fixed, isolating the downstream methodology from selection scatter.

One scoring rule deserves its own statement, because the data forced it. For decisions with *no answer key*, agreement between arms is a *concordance* measure and must never be reported as accuracy: neither arm can be wrong, a divergence is not an error, and any benchmark number that scores it as one is invalid. Background-interval placement is the proving case. For one burst that triggered about two minutes$^{(p)}$ after a radiation-belt exit, two equally defensible uncontaminated baselines imply excesses in the same interval that differ by a factor of seven$^{(p)}$ (+68 versus +481 counts s^{-1}): the excess is real while its magnitude is not a measurement, and the published literature uses this very burst

as its showcase that polynomial background choices can be ambiguous (Biltzinger et al. 2020). The benchmark therefore separates its dimensions: rule compliance and downstream impact are truth-bearing and scored; window agreement is reported as concordance only. The separation expires only if a physical background model replaces the polynomial one, creating the answer key that does not currently exist.

9. WALKTHROUGH-ERA RESULTS

All numbers in this section are walkthrough-era and provisional$^{(p)}$; they will be regenerated by the frozen production sweep. The full 106-burst sample has passed once through the fitting engine, with one burst excluded for response-matrix time coverage and reported as an exclusion rather than silently dropped$^{(p)}$. No cross-burst census sentence is permitted yet: a ledgered retry debt concentrated in the multi-break and composite model families must reach a terminal state first, and per the doctrine that debt is visible in the products, never normalized away. Among the walked bursts: a burst whose apparent two-break "decisive" margin traced to a railed reference model and now carries a capped-bound stamp; a burst whose published photospheric identification our wider menu reduces to a forced choice between statistically equivalent thermal and non-thermal descriptions; a burst in which the fitted low-energy break and the fitted blackbody peak track each other at the few-percent level across eleven bins—one spectral feature wearing two model names; the campaign's first soft burst (the L27 episode), whose positive low-energy slopes are carried entirely by one-break continua; the campaign's first intensity-tracking burst, whose blind $E_{\rm p}$ track follows the flux and whose late-time cutoff evolution sits in open, documented tension with a published constant-Γ curvature interpretation; and the field's best-known polarimetric burst, for which published polarization-angle rotations enter the frozen predictions as constraints from an instrument we do not fit. The architecture itself is under a live acceptance test: the entire law—skills, roster, gates, skeleton—was handed to a fresh session with no inherited conversational context, on the argument that an architecture that works only for its authors is not yet an architecture. In that arm's first burst, the first seven of the eleven gated steps have passed their human gates with every approval stamped$^{(p)}$—each stamp carrying its own provenance, including one approval given colloquially and typed as read-as-approval, and one recorded late, the case below—and the mechanisms, not the operators' memories, caught what needed catching. The approval ledger's cascade exposed an approval that existed only in conversation for seven hours$^{(p)}$ and forced it onto the record with the delay disclosed. The data-inventory contract surfaced a hundred-millisecond telemetry stall$^{(p)}$ invisible to the standard exposure accounting (its count deficit is cancelled by a buffered catch-up in the following readout bin), and the repair was accepted only under a robustness criterion: the chosen background cut is the one whose fit quality is independent of the binning grid's phase, which otherwise moves the fit statistic by up to a factor of $2.4^{(p)}$. And a background window found overlapping the approved source window by about thirty-two seconds$^{(p)}$ was measured as a real excess ($\geq 3.3\sigma$ in every fit variant$^{(p)}$), repaired by an explicit human ruling, and its residual tail contamination carried forward as a disclosed systematic rather than silently accepted. Each catch was typed to its failure class and distilled; the steps beyond the stamped gates remain open and contribute no accepted numbers here.

The Campaign Ledger records, per burst: lessons born, defects found, blind-prediction outcomes, and human gate corrections; it is released with the paper, and Fig. 4 is computed from it. External, zero-cost validations accumulated along the way: our adaptive binning reproduces a published analysis' Bayesian-block edges to three decimal places on a shared burst (Li & Zhang 2021), and twenty-two of our sample's bursts carry published lag, width, and evolution-class measurements (Lu et al. 2018) against which the temporal chain is validated burst by burst as the campaign reaches them.

10. DISCUSSION

10.1. *Failure taxonomy*

Every mode below was observed in this campaign, caught, and converted into doctrine; none was caught by the failing agent itself. *Over-claiming*: three retracted claims, each computed from technically correct numbers whose validity flags had not been checked; caught by the human gate and by an independent agent audit. *Prescribed unanimity*: a panel's unanimous verdict traced verbatim to the shared prior document (§6). *Confabulation*: a reviewer channel reporting statistics absent from its own assigned figure. *Estimator mismatch*: an ad-hoc quality check using a simpler background model than production, manufacturing a 5.6σ alarm against a selection a human had already, correctly, approved—the rule is that a check must use the production estimator or it is not a check. *Tool-interface drift*: invoking one's own tools with invented arguments. *Momentum skips*: advancing past a presentation step the Protocol requires; caught by the human, converted into an explicit rule. *Diagnostic-becomes-result drift*: treating a data-dropped sensitivity refit as evidence (§6).

The taxonomy is the practical answer to "where does the agent still need a human": at every point where the system's own priors, momentum, or plausible-but-unchecked numbers are the failure mode, because those are precisely the failures self-review does not catch.

10.2. *The capability boundary*

The same experiments that expose these failures locate the boundary positively: differently-primed AI reviewers are strong *generators* of checkable findings—several entered the engine as permanent guards—and weak *adjudicators* of contested verdicts. We therefore spend agency where generation pays (proposing, localizing, refuting, attributing) and reserve adjudication for the gated human, with the audit trail as the interface between them.

10.3. *Outlook: the discovery plane, and creativity as graph operations*

The production plane described in this paper deliberately excludes open-ended discovery; a second, sandboxed plane exists for it, governed by one rule: the agent may invent a new scorecard after seeing the game, but may not pretend the scorecard was chosen before the game. What would make that plane *generative* rather than merely permitted? We close with a hypothesis we are developing and have begun to stress-test: that creative scientific acts are, predominantly, typed operations on the knowledge graph of the literature—$\mathbf{A'}$, drawing an edge along a long, archivally stale, or never-co-cited path between semantically compatible regions; $\mathbf{B'}$, negating a *named, load-bearing assumption* of a specific published derivation, as the first generative move rather than as a consequence of new data; and $\mathbf{C}$, constructing a new graph element outright—a phenomenon-node, a question, a topology of existing components. An adversarial audit of this taxonomy against twenty-four historically recognized discoveries found the two-class version (A′, B′) insufficient —class C is required, and the classification must apply to *acts*, not results (a landmark observation can contain no creative jump)—while identifying a genuinely open architectural slot: a literature-scale knowledge graph in which each result carries its derivation assumptions as first-class metadata, with negation as a provenance-tracked generative operator (Boden 2004; Swanson 1986; Uzzi et al. 2013; Foster et al. 2013; Wiggins 2006). The Reconciliation Protocol already extracts, burst by burst, exactly the assumption-level statements such a graph requires; the discovery plane is where they will compound.

10.4. *What generalizes*

Gates with stamps, lessons-as-tests, blind-first frozen predictions, frame-alignment before any diff, independence-at-the-primitive, and the failure taxonomy are domain-agnostic and export as a package; the engine, the menu, and the lessons' content are the domain deposit that makes the package worth executing. Both are released. The package is, in the vocabulary now common for such systems, a *domain harness*: the Library, the Register, the roster contracts, and the doctrine are text and transfer unchanged across vendor agent platforms, while the enforcement layer—hooks, state derivation, cascades—is rebuilt per platform. We treat that portability as an experiment rather than an assumption: the same law executed by a second vendor's agent on a known burst, compared gate by gate, is the designed test, and it is pending.

11. CONCLUSIONS

Yes—as a gated workflow with bounded agency, trained harness-side under verifiable lessons, and audited by a doctrine that treats agreement with suspicion and refutation as a service. The system walks bursts end to end under stamped gates, has taught itself thirty-two durable lessons$^{(p)}$, found its own engine defects including one census-grade selection bias, validated a known defect against published ground truth, refuted one of its own frozen predictions, and now runs as written in a fresh session with no inherited context—the acceptance test of §9. The per-burst numbers are provisional by design; the Skill Library is not. When the learning curve flattens, the pipeline freezes and runs the full sample once; that sweep, not this paper, produces the census. The question in the title has an honest answer only with its architecture attached—and that architecture, we argue, is the contribution.

We do not mistake the current boundary for a permanent one. Every failure mode in §10.1 was caught at a gate, and every catch became a lesson: the gates are where human judgment is *transferred*, not merely exercised, and the Library is the accumulating transfer. The claim that some capability—taste in ambiguous selections, suspicion of one's own agreement, the adjudication of contested verdicts—is inherently human is, under this architecture, an empirical claim about the tail of a learning curve, testable by whether the gate-correction rate falls to zero as lessons accumulate. The Divergence Ledger of §5.1 makes that test operational: the convergence rate between the agent's proposal and the human's decision is now a defined, recorded quantity, accumulating with every gate$^{(p)}$. And the curriculum need not stop at analysis: the discovery-plane operators of §10.3—bridging stale regions of the literature graph,

negating named load-bearing assumptions, constructing new observables—are mechanisms by which discovery itself becomes teachable. Whether the gates ever go fully quiet is not something this paper can assert; it is something this campaign, continued to convergence and beyond, is built to measure. The gate, in the end, is a curriculum—and curricula are meant to be completed.

APPENDIX

A. COMPONENT ROSTER

Table 2 lists every component of the harness of §3 with its job in one sentence, its kind, how it executes, and its status at the census.

Table 2. The component roster: the work of each component in one sentence. Kind: guide / sensor / action / store / producer / actor / boundary; execution: computational (C), inferential (I), human (H). Status is what is on disk at commit `4df6884` (census 2026-09-02): DEPLOYED = a file or product exists; PROPOSED = a register row or law-file actor without one. Origins are quoted in the source table.

component	job	kind	exec.	status
the loop (run the reasoning loop)				
Mode-B harness session (Claude Code plays the roster)	Runs the reasoning loop in a subscription coding harness, consuming the agent files, hooks and skills natively while humans gate in-session.	actor	I	DEPLOYED
Burst-walkthrough protocol + fresh-session boot ritual	Opens every session from the law files and the disk board, then presents each of the eleven steps to a gate before the next runs.	guide	I/H	DEPLOYED
Dispatcher (A2)	Reads the task and the register, then returns the agents, gates and order required, naming every guard that is still unbuilt.	actor (planner)	I	DEPLOYED
Queue manager + workflow set (A17; `wf-*`)	Would own campaign sequencing: read every burst state, run the next legal transition as a typed workflow, classify failures, resume after kill.	actor + action	C	PROPOSED
Orchestrator chains (prototype transitions)	Run fit, merge, products, temporal, assembly and retry stages as resumable shell/python chains; none invokes an agent.	action	C	DEPLOYED
tools (execute tools: the deterministic producers)				
Spectral fitting engine (`scripts/10`)	Fits every model in the menu jointly to each Bayesian block and the integrated window, writing validity, rail and AIC columns per model.	producer	C	DEPLOYED
Bayesian-block binning (`scripts/27b`)	Cuts each burst into Bayesian blocks on the brightest NaI and merges blocks below the significance floor, using 3ML's own classes.	producer	C	DEPLOYED
Stage-1 approval (`scripts/39` GUI + ingest → stamped catalog)	Renders light curves, takes a human or AI decision on detectors, background and source windows, and stamps who approved what and how.	action + store	C/I/H	DEPLOYED
Step figures + product manifest (`scripts/44`, `45`)	Draws one figure per pipeline step and prints a manifest of every product made or missing, so absence is stated.	producer	C	DEPLOYED
Temporal chain (`scripts/40`, `46`, `47c`)	Measures T90, MVT and spectral lag from the same approved selections, importing the PI's validated lag code unmodified.	producer	C	DEPLOYED
SED product family (`scripts/41c`, `41d`, `41e`)	Renders each bin's νF_ν figure under XSPEC semantics, plots parameter evolution, and composites all-model montages, refusing any panel that disagrees with the stored fit.	producer (with code guards)	C	DEPLOYED
Per-burst report (`scripts/48`)	Assembles one document per burst with captions written from that burst's numbers and every honest flag: ties, edge classes, missing products.	producer	C	DEPLOYED

Table 2 *continued*

Table 2 *(continued)*

component	job	kind	exec.	status
context (assemble and manage context)				
Per-step skill files (10, indexed by the ledger)	Carry each step's criteria, checklists and numbered lessons, read end-to-end at every step open.	guide	I	DEPLOYED
Cross-cutting contracts (S-items, R-items, shipping, referee, figure style)	State, in the PI's quoted words, what every figure, report, shipment and referee round must satisfy.	guide (contract)	I	DEPLOYED
Law files, orientation and memory	Fix the principles, the roster, the state machine and the boot order; orient any agent to the repo; carry cross-session lessons.	guide	I	DEPLOYED
Skill-reader (A1)	Reads the step's skill file, ledgers and open register rows, and returns the binding checklist for this burst; produces nothing else.	actor (checklist-maker)	I	DEPLOYED
Prior-art-reader (A13)	Sweeps this repo and the project family for existing proofs before any root-cause or redo, and says what is already answered.	sensor	I	DEPLOYED
state (persist state)				
State machine + disk-derived board (`dev/agent_state.py`)	Derives every burst's state from evidence on disk, never from memory; the skeleton file, not the code, declares each failure class's behaviour.	store + guide	C	DEPLOYED
Live report, approval stamps, invalidation cascade	Assembles the per-burst live document, records identity-bound stamps for every step, and clears downstream markers when an approved step changes.	action + store	C/H	DEPLOYED
Provenance sidecars, promotion receipts, commit pin, reproduction record, two-root index	Binds every product to its inputs and generator by hash, pins generators by commit, and resolves which of two product roots is canonical.	store	C	DEPLOYED
Object schemas + schema test	Declares the shape of ten object types already written and fails the suite when any instance does not validate, four frozen files excepted.	store (typed) + sensor	C	DEPLOYED
Provenance stamp + action trace	Appends one JSON line per regulated action, validated when jsonschema is importable, naming the model, harness, session and git head; "unknown" is honest.	store + sensor	C	DEPLOYED
Verdict and reconciliation ledgers	Keeps every sha-bound figure verdict, frozen blind prediction and literature reconciliation on disk where the hooks and readers look.	store	C/I	DEPLOYED
First-class actions (present, approve, promote, quarantine, invalidate, launch-producer, deliver) with intent → preflight → commit → receipt	Would wrap every state-changing verb as intent, preflight, commit and receipt, with an actor and model on each; two verbs exist today.	action	C	PROPOSED
boundaries (enforce boundaries: hooks, roles, verifiers, the release bar)				
No-ship hook (E1)	Blocks delivery of any PNG whose sha256 appears in no VISION_QC ledger.	sensor (hook)	C	DEPLOYED
Dispatch hook (E2)	Blocks a pipeline-producer launch unless a dispatch plan younger than 24 h exists on disk.	sensor (hook)	C	DEPLOYED
Raw-data guard	Blocks any shell command that deletes or overwrites under `data/bn*`, unless it is the acquisition path.	sensor (hook)	C	DEPLOYED
RAM arbiter + watchdog (E3)	Admits heavy jobs by gigabytes against measured peak RSS through one machine-wide semaphore, and brakes admission when the machine pages.	boundary (resource gate)	C	DEPLOYED
Roles, tool grants and the PI gate (approver, A16)	Separates producer, verifier and approver; the PI approves every step; tool grants are per agent file, Bash-stripping for three readers still undecided.	boundary (roles) + actor (human)	C/H	DEPLOYED
Figure-verifier (A8)	Inspects every figure in a fresh context against the standing contract and its same-run sidecar, returning a verdict the orchestrator sha-binds.	sensor	I	DEPLOYED
Numbers-verifier (A9)	Recomputes every printed number from the run's own tables, sidecars and catalogs, and fails loudly on anything untraceable.	sensor	C/I	DEPLOYED

Table 2 *continued*

Table 2 *(continued)*

component	job	kind	exec.	status
Seed-auditor (A5)	Finds the seed of every stochastic product, traces it into every RNG, and, where cheap, reruns the smallest unit to prove bit-identical replay.	sensor	C/I	DEPLOYED
Admission-gate (A4)	Screens every row bound for a committed catalog for type, identity and cross-field sanity, refusing or flagging rows with undisclosed rails.	sensor	C/I	DEPLOYED
Port-verifier (A12)	Runs a ported function and its source code on a synthetic case and refuses the port unless they agree numerically.	sensor	C/I	DEPLOYED
Report-conformance gate (A10)	Would verify an assembled deliverable against the report contract, including sha equality between report, figures and the promoted table.	sensor	I	PROPOSED
Queued boundary build (caps, spend ledger, extra hook points, pre-commit, sandbox)	Would cap verifier rounds, log paid spend, auto-trace every tool call, run the fast suite pre-commit, and sandbox producers.	boundary	C	PROPOSED
the steering loop (learning without gradients)				
Agent-requirements register (NR rows)	Records, one row per needed guard or agent, the incident that bore it and its status, so every piece of the machine carries its why.	store + guide	I	DEPLOYED
Distiller (A14)	Root-causes each incident to its primitive, places the countermeasure at the strongest feasible layer, and writes the lesson and register row the same session.	actor (writer)	I	DEPLOYED
Test suite (lessons-as-tests) + hand-run QC scripts	Turns every invariant-shaped lesson and catalog rule into a check over the real products, as a pytest or a hand-run script.	sensor	C	DEPLOYED
ID guards (register ids + lesson prefixes)	Fails the suite on a duplicate register id, a row outside the table, a dangling citation, or a lesson prefix shared by two files.	sensor	C	DEPLOYED
Eval battery (one command)	Runs the suite and the fit-table auditor over every promoted table in one command and records the result with model id and git head.	sensor (eval harness)	C	DEPLOYED
External auditor, two hats (A15: Codex supervisor / blind referee panel T0–T2)	Reviews milestones either as a full-context supervisor or as three blind referees with fixed temperaments, never both on one product version.	sensor	I	DEPLOYED
Divergence learner	Elicits, in the human's words, why a human choice differed from the machinery when neither failed, and ledgers whether the reason generalises.	guide + store	I/H	DEPLOYED
science-specific sensors (the numbers discipline)				
Fit-table auditor + count-coordinates rule	Recomputes tie sets, adopted models, chain-gate verdicts, rail census and AIC offset from the engine's columns, then checks every presented count carries its three coordinates.	sensor (code) + guide (rule)	C	DEPLOYED
Tie-reporter (A6)	Recomputes ΔAIC heads from stored AICs and returns, for any bin within the tie threshold, the tie set, the offending claim and the rewording required.	sensor	C/I	DEPLOYED
Edge and mismatch stamps carried on the artifact	Stamps an edge-constrained component, a bound-pinned parameter, or a panel that disagrees with the engine on the artifact itself, never in prose alone.	sensor (artifact layer)	C	DEPLOYED
Preference census with validity gate (`dev/model_preference.py`)	Computes the tracked-model construct per burst from stored AICs, ranking only over fits the engine marked valid and refusing to write on mismatch.	sensor + producer	C	DEPLOYED
Notes-reviewers (A7) + literature agent (A11)	Read residuals bin by bin in fresh contexts and attribute every literature mismatch only after our numbers are frozen; both run as session protocol.	sensor	I	DEPLOYED
Queued science-guard code	Would ground the engine on synthetic truth, refuse stale temporal columns, fail closed on vacant read paths, and refuse ancestor-less winners in engine and report.	sensor	C	PROPOSED

B. REPRODUCIBILITY

The release comprises the Skill Library, the engine, the Lesson Test-Suite, the Campaign Ledger, and the Burst Records, archived with a DOI **[PENDING run: Zenodo concept DOI at release]**. A fresh clone cannot silently reproduce our fits: analysis products are regenerated, not shipped, so reproduction begins at the binning step from tracked inputs; six sample bursts include tracked raw data. The test suite runs in a base scientific Python environment; the fitting engine requires the standard *Fermi* analysis stack.

C. ADVERSARIAL VERIFICATION PROTOCOL

The protocol: independent reviewer channels are assigned disjoint evidence primitives and briefed adversarially; an adjudicator with full access audits every channel's claims against the primary artifacts; adversary claims are themselves verified by rerun. Results of its first full execution appear in §6. A standing per-burst refuter pass—one honest-refuter agent attacking each Burst Record's verdict before it closes, its surviving objections recorded in the Record—is the protocol's next planned extension. **[PENDING run: refuter pass: pending approval and first execution]**

REFERENCES

Anthropic. 2024, Building Effective AI Agents, https://www.anthropic.com/engineering/building-effective-agents

Biltzinger, B., Kunzweiler, F., Greiner, J., Toelge, K., & Burgess, J. M. 2020, A&A, 640, A8, doi: 10.1051/0004-6361/201937347

Boden, M. A. 2004, The Creative Mind: Myths and Mechanisms, 2nd edn. (London: Routledge)

Busby, I., & Lazzati, D. 2024, ApJ, 972, 83, doi: 10.3847/1538-4357/ad6564

Fana Dirirsa, F., Razzaque, S., Piron, F., et al. 2019, ApJ, 887, 13, doi: 10.3847/1538-4357/ab4e11

Foster, J. G., Rzhetsky, A., & Evans, J. A. 2013, arXiv e-prints, arXiv:1302.6906, doi: 10.48550/arXiv.1302.6906

Guo, D., Yang, D., Zhang, H., et al. 2025, Nature, 645, 633, doi: 10.1038/s41586-025-09422-z

Li, L., & Zhang, B. 2021, ApJS, 253, 43, doi: 10.3847/1538-4365/abded1

Lu, R.-J., Liang, Y.-F., Lin, D.-B., et al. 2018, ApJ, 865, 153, doi: 10.3847/1538-4357/aada16

Shinn, N., Cassano, F., Berman, E., et al. 2023, arXiv e-prints, arXiv:2303.11366, doi: 10.48550/arXiv.2303.11366

Siddique, I., Sajjad, S., & Motiwala, K. 2022, ApJ, 938, 159, doi: 10.3847/1538-4357/ac8d05

Swanson, D. R. 1986, Perspectives in Biology and Medicine, 30, 7, doi: 10.1353/pbm.1986.0087

Uzzi, B., Mukherjee, S., Stringer, M., & Jones, B. 2013, Science, 342, 468, doi: 10.1126/science.1240474

Villaescusa-Navarro, F., Bolliet, B., Villanueva-Domingo, P., et al. 2025, arXiv e-prints, arXiv:2510.26887, doi: 10.48550/arXiv.2510.26887

Wang, G., Xie, Y., Jiang, Y., et al. 2023, arXiv e-prints, arXiv:2305.16291, doi: 10.48550/arXiv.2305.16291

Wiggins, G. A. 2006, Knowledge-Based Systems, 19, 449, doi: 10.1016/j.knosys.2006.04.009

Yamada, Y., Tjarko Lange, R., Lu, C., et al. 2025, arXiv e-prints, arXiv:2504.08066, doi: 10.48550/arXiv.2504.08066